

## Assignment(8)

Name: kasuri Rohith

Batch: 23

Roll no: 2303a510i2

### Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

- Requirements:

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.
- o No spaces allowed.

#### Example Assert Test Cases:

```
assert is_valid_username("User123") == True
```

```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

#### Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

Code:

```

1  def is_valid_username(username):
2      if len(username) < 5 or len(username) > 15:
3          return False
4      if username[0].isdigit():
5          return False
6      if " " in username:
7          return False
8      if not username.isalnum():
9          return False
10     return True
11
12     assert is_valid_username("User123") == True
13     assert is_valid_username("12User") == False
14     assert is_valid_username("Us er") == False
15
16     print("Username all tests passed")

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

python3.11.exe c:/Users/rohit/Links/training.py
Username all tests passed
PS C:\Users\rohit\Links>

```

Task Description #2 (Even–Odd & Type Classification – Apply

AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.
- Requirements:
  - o If input is an integer, classify as "Even" or "Odd".
  - o If input is 0, return "Zero".
  - o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

Code:

```
17
18 def classify_value(x):
19     if type(x) != int:
20         return "Invalid Input"
21     if x == 0:
22         return "Zero"
23     if x % 2 == 0:
24         return "Even"
25     return "Odd"
26
27 assert classify_value(8) == "Even"
28 assert classify_value(7) == "Odd"
29 assert classify_value("abc") == "Invalid Input"
30 print("all tests passed")
31
```

Output:

```
PS C:\Users\rohit\Links> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/Links/training.py
all tests passed
PS C:\Users\rohit\Links>
```

Task Description #3 (Palindrome Checker – Apply AI for

String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.
- Requirements:

- o Ignore case, spaces, and punctuation.
- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
```

```
assert is_palindrome("A man a plan a canal Panama") ==
```

True

```
assert is_palindrome("Python") == False
```

Expected Output #3:

- Function correctly identifying palindromes and passing all

AI-generated tests.

Code:

```
33 def is_palindrome(text):
34     cl = ""
35
36     for ch in text.lower():
37         if ch.isalnum():
38             cl += ch
39
40     return cl == cl[::-1]
41
42     assert is_palindrome("Madam") == True
43     assert is_palindrome("Python") == False
44     assert is_palindrome("") == True
45
46     print("palindrome tests passed")
47
```

Output:

```
PS C:\Users\rohit\Links> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/Links/training.py
palindrome tests passed
PS C:\Users\rohit\Links>
```

#### Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

- Requirements:

- o Must contain @ and .

- o Must not start or end with special characters.

- o Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

```
assert validate_email("userexample.com") == False
```

```
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

Code:

```
49  def validate_email(email):
50      if "@" not in email or "." not in email:
51          return False
52      if email[0] in "@." or email[-1] in "@.":
53          return False
54      return True
55
56  assert validate_email("user@example.com") == True
57  assert validate_email("userexample.com") == False
58  assert validate_email("@gmail.com") == False
59
60  print("email tests passed")
61
```

Output:

```
PS C:\Users\rohit\Links> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/Links/training.py
email tests passed
PS C:\Users\rohit\Links>
```

#### Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).

- Test Cases to Design:

- o Normal case: 6 → True, 10 → False.

- o Edge case: 1.

- o Negative number case.

- o Larger case: 28.

- Requirement: Validate correctness with assertions.

#### Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).

- Test Cases to Design:

- o Normal case: 12 → True, 15 → False.

- o Edge case: 1.

- o Negative number case.

- o Large case: 945.

Requirement: Validate correctness with unittest

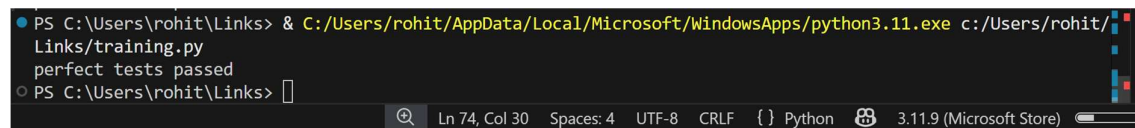
Code:

```

62  ✓ def is_perfect(n):
63  ✓     if n <= 1:
64      |         return False
65      |         total = 0
66  ✓     for i in range(1, n):
67  ✓     |         if n % i == 0:
68      |         |         total += i
69      |         return total == n
70
71  assert is_perfect(6) == True
72  assert is_perfect(10) == False
73  assert is_perfect(28) == True
74  print("perfect tests passed")

```

Output:



```

PS C:\Users\rohit\Links> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/Links/training.py
perfect tests passed
PS C:\Users\rohit\Links>

```

#### Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).
- Test Cases to Design:
  - o Normal case: 12 → True, 15 → False.
  - o Edge case: 1.
  - o Negative number case.
  - o Large case: 945.

Requirement: Validate correctness with unittest

Code:

```

76 def is_abundant(n):
77     total = 0
78     for i in range(1, n):
79         if n % i == 0:
80             total += i
81     return total > n
82
83 import unittest
84
85 class TestAbundant(unittest.TestCase):
86     def test_cases(self):
87         self.assertTrue(is_abundant(12))
88         self.assertFalse(is_abundant(15))
89         self.assertTrue(is_abundant(945))
90
91 if __name__ == "__main__":
92     unittest.main()
93

```

Output:

```

PS C:\Users\rohit\Links> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/Links/training.py
-----
Ran 1 test in 0.000s
OK
PS C:\Users\rohit\Links>

```

### Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).

- Test Cases to Design:

- o Normal case: 8 → True, 12 → False.

- o Edge case: 1.

- o Negative number case.

- o Large case: 546.

Requirement: Validate correctness with pytest.

Code;



```

56 ∨ def is_deficient(n):
57     total = 0
58     for i in range(1, n):
59         if n % i == 0:
60             total += i
61     return total < n
62
63 ∨ def test_is_deficient():
64     assert is_deficient(8) == True
65     assert is_deficient(12) == False
66     assert is_deficient(546) == False

```

Output:

```

PS C:\Users\rohit\Links> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/Links/demo8.py
PS C:\Users\rohit\Links> python -m pytest demo8.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\rohit\Links
plugins: anyio-3.7.1
collected 1 item

demo8.py . [100%]

===== 1 passed in 0.11s =====
PS C:\Users\rohit\Links>

```

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

code:

```

69  ✓ def leap_year_checker(year):
70  ✓     if year % 400 == 0:
71      |         return True
72  ✓     if year % 100 == 0:
73      |         return False
74  ✓     if year % 4 == 0:
75      |         return True
76      |         return False
77
78  ✓ def test_leap_year():
79      |     assert leap_year_checker(2000) == True
80      |     assert leap_year_checker(1900) == False
81      |     assert leap_year_checker(2024) == True
82      |     assert leap_year_checker(2023) == False
83

```

Output:

```

• PS C:\Users\rohit\Links> python -m pytest demo8.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\rohit\Links
plugins: anyio-3.7.1
collected 1 item

demo8.py . [100%]

===== 1 passed in 0.32s =====
❖ PS C:\Users\rohit\Links>

```

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

Code:

```

85  def sum_of_digits(n):
86      total = 0
87      for d in str(abs(n)):
88          total += int(d)
89      return total
90
91  def test_sum_digits():
92      assert sum_of_digits(123) == 6
93      assert sum_of_digits(0) == 0
94      assert sum_of_digits(-456) == 15
95      assert sum_of_digits(999) == 27
96      assert sum_of_digits(1001) == 2
97      assert sum_of_digits(7) == 7
98      assert sum_of_digits(-100) == 1
99

```

Output:

```

PS C:\Users\rohit\Links> python -m pytest demo8.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\rohit\Links
plugins: anyio-3.7.1
collected 1 item

demo8.py . [100%]

===== 1 passed in 0.06s =====
PS C:\Users\rohit\Links>

```

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

code:

```

101 def sort_numbers(arr):
102     arr = arr.copy()
103     for i in range(len(arr)):
104         for j in range(len(arr) - 1):
105             if arr[j] > arr[j + 1]:
106                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
107     return arr
108
109 def test_sort():
110     assert sort_numbers([3, 2, 1]) == [1, 2, 3]
111     assert sort_numbers([5, 1, 4]) == [1, 4, 5]
112     assert sort_numbers([1]) == [1]
113     assert sort_numbers([]) == []

```

Output:

```

PS C:\Users\rohit\Links> python -m pytest demo8.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\rohit\Links
plugins: anyio-3.7.1
collected 1 item

demo8.py . [100%]

===== 1 passed in 0.15s =====
PS C:\Users\rohit\Links>

```

Task 11 :

Write a function ReverseString and validate its implementation

using 5 unittest test cases

code:

```

115 def reverse_string(text):
116     return text[::-1]
117
118 import unittest
119
120 class TestReverse(unittest.TestCase):
121     def test_cases(self):
122         self.assertEqual(reverse_string("hello"), "olleh")
123         self.assertEqual(reverse_string("a"), "a")
124         self.assertEqual(reverse_string("123"), "321")
125
126 if __name__ == "__main__":
127     unittest.main()
128

```

Output:

```
PS C:\Users\rohit\Links> python -m pytest demo8.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\rohit\Links
plugins: anyio-3.7.1
collected 1 item

demo8.py . [100%]

===== 1 passed in 0.06s =====
PS C:\Users\rohit\Links>
```

Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

Code:

```
130 def anagram_checker(a, b):
131     return sorted(a.lower()) == sorted(b.lower())
132
133 import unittest
134
135 class TestAnagram(unittest.TestCase):
136     def test_cases(self):
137         self.assertTrue(anagram_checker("listen", "silent"))
138         self.assertFalse(anagram_checker("hello", "world"))
139         self.assertTrue(anagram_checker("race", "care"))
140
141 if __name__ == "__main__":
142     unittest.main()
143
```

Output:

```
PS C:\Users\rohit\Links> python -m pytest demo8.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\rohit\Links
plugins: anyio-3.7.1
collected 1 item

demo8.py . [100%]

===== 1 passed in 0.14s =====
PS C:\Users\rohit\Links>
```

Task 13 :

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

Code:

```
162 def armstrong_checker(n):
163     total = 0
164     power = len(str(n))
165     for d in str(n):
166         total += int(d) ** power
167     return total == n
168
169 import unittest
170
171 class TestArmstrong(unittest.TestCase):
172     def test_cases(self):
173         self.assertTrue(armstrong_checker(153))
174         self.assertFalse(armstrong_checker(10))
175         self.assertTrue(armstrong_checker(370))
176
177 if __name__ == "__main__":
178     unittest.main()
179
```

Code:

```
PS C:\Users\rohit\Links> python -m pytest demo8.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\rohit\Links
plugins: anyio-3.7.1
collected 1 item

demo8.py . [100%]

===== 1 passed in 0.09s =====
❖ PS C:\Users\rohit\Links>
```